Worksheet — 1.1 Processors, Input, Output & Storage

OCR A Level Computer Science (H446) — Component 01, Section 1.1 (1.1.1 processor, 1.1.2 processor types, 1.1.3 I/O & storage).

Name: __ **Date:** ____

Time: ~30 min. Write your answers in the spaces; check against the answer key at the end.

1. Key-term match

Match each term (1–8) to its definition (A–H). Write the letter on the line.

Terms

1. Register ..... ____
2. Cache ..... ____
3. Pipelining ..... ____
4. Core ..... ____
5. Volatile ..... ____
6. Virtual memory ..... ____
7. GPU ..... ____
8. Control Unit ..... ____

Definitions

- **A.** Overlapping the fetch, decode and execute of consecutive instructions to raise throughput.
- **B.** Loses its contents when power is removed.
- **C.** Small, very fast store inside the CPU that holds data during processing.
- **D.** Many-core processor that performs the same operation on large data sets in parallel.
- **E.** Use of secondary storage as extra RAM when physical RAM is full.
- **F.** An independent processing unit able to run its own fetch–decode–execute cycle.
- **G.** Small, fast memory on or near the CPU holding frequently used data/instructions.
- **H.** Decodes instructions, generates control signals and synchronises the CPU with the clock.

2. Fill in the blanks — the Fetch–Decode–Execute cycle

Complete the passage using the word bank. Each word is used once.

Word bank: opcode · operand · PC · MDR · CIR · data · control · increment · ALU · address

During **fetch**, the address of the next instruction is copied from the __ **into the MAR. A read signal is sent on the _ *bus, and the instruction returns along the* _ bus into the __. The processor then _*s the PC so it points to the next instruction, and the fetched instruction is copied from the MDR into the* _.

During **decode**, the Control Unit splits the instruction into the __ **(the operation to perform) and the** _ ***(the data or*** _ __ to act on).

During **execute**, the instruction is carried out — for example a calculation is performed by the _____ with the result stored in the accumulator.

3. State the register

For each description, name the register (PC, ACC, MAR, MDR or CIR).

a) Holds the address of the **next** instruction to be fetched. ..... _____

b) Holds the **address** of the location currently being read from or written to. ..... _____

c) Holds the **data or instruction** just fetched from (or about to be written to) memory. ..... _____

d) Holds the **current instruction** while it is being decoded and executed. ..... _____

e) Stores the **immediate result** of an ALU calculation. ..... _____

4. Put the FDE steps in order

Number these fetch-stage steps 1–4 in the order they happen.

____ Read signal placed on the control bus; instruction returns via the data bus into the MDR.

____ The contents of the MDR are copied into the CIR.

____ The address in the PC is copied into the MAR.

____ The PC is incremented (PC := PC + 1).

5. Short answer — performance & architectures

a) State **two** factors that affect CPU performance and explain the effect of each.

b) "Adding more cores always makes a computer faster." Give the **caveat** that makes this statement only partly true.

c) Explain the key difference between **Von Neumann** and **Harvard** architectures, and give one typical use of each.

d) Give **two** differences between **CISC** and **RISC** processors.

e) Explain why a GPU is well suited to graphics and machine-learning work but poor at branch-heavy sequential tasks.

6. Storage device selection table

Complete the empty cells. For each storage type, state **how it works**, **one pro**, **one con** and **one example use**.

Type	How it works	One pro	One con	Example use
Magnetic (e.g. HDD, tape)				
Flash/SSD (e.g. SSD, USB, SD)				
Optical (e.g. CD, DVD, Blu-ray)				

7. Challenge box

> **Challenge — virtual memory & thrashing.** A user keeps many large applications open and the computer becomes very slow, with the hard disk constantly active.
>
> (i) Explain what **virtual memory** is and why the OS is using it here.
>
> (ii) Name the problem causing the slowdown and explain **why** moving pages between RAM and disk is so slow.
>
> (iii) Suggest **one** hardware change that would reduce the problem, and justify it.

Answer key

1. Key-term match 1–C, 2–G, 3–A, 4–F, 5–B, 6–E, 7–D, 8–H.

2. Fill in the blanks (FDE) …copied from the **PC** into the MAR. A read signal is sent on the **control** bus, and the instruction returns along the **data** bus into the **MDR**. The processor then **increment**s the PC… copied from the MDR into the **CIR**. Decode: splits the instruction into the **opcode** (the operation) and the **operand** (the data or **address**). Execute: a calculation is performed by the **ALU**, result stored in the accumulator.

3. State the register a) PC b) MAR c) MDR d) CIR e) ACC

4. Put the FDE steps in order 1 — The address in the PC is copied into the MAR. 2 — Read signal placed on the control bus; instruction returns via the data bus into the MDR. 3 — The PC is incremented (PC := PC + 1). 4 — The contents of the MDR are copied into the CIR. *(Steps 3 and 4 may be quoted either way round; the key points are that the PC must be incremented and the MDR must be copied into the CIR.)*

5. Short answer — performance & architectures a) Any two, e.g.: - **Clock speed** — pulses per second synchronise the CPU; a higher clock speed means more FDE cycles per second (limited by heat/power). - **Cores** — each core has its own FDE cycle, so multiple instructions can be processed simultaneously. - **Cache** — small, very fast memory on/near the CPU holding frequent/recent data, reducing slow fetches from RAM. - **Pipelining** — overlaps fetch/decode/execute of consecutive instructions, increasing throughput. b) Extra cores only help **if the task can be split into parallel parts and the software is written to use multiple cores**; inherently sequential tasks gain little. c) **Von Neumann** uses one shared memory and bus for both data and instructions (simpler/cheaper but causes a bottleneck because it cannot fetch an instruction and read/write data at the same time); **Harvard** uses separate memories/buses for data and instructions, allowing simultaneous access. Use: Von Neumann — general-purpose PCs/laptops; Harvard — embedded systems / microcontrollers / DSPs. d) Any two, e.g.: CISC has many complex, variable-length, multi-cycle instructions with complexity in hardware; RISC has few simple fixed-length instructions aiming for one per cycle with complexity in the compiler. RISC pipelines more easily and uses less power (good for battery devices/ARM); CISC is typical of x86 desktops. e) A GPU has thousands of small cores that apply the **same operation to many data items in parallel (data parallelism)**, which suits pixels/vertices and large ML matrices. Branch-heavy, dependency-laden or inherently sequential work cannot be parallelised this way (each step needs the previous result), so a CPU handles it better.

6. Storage device selection table (example answers)

Type	How it works	One pro	One con	Example use
Magnetic	Magnetised patterns/polarity on a spinning platter or tape, read/written by a R/W head	Huge capacity, low cost per GB	Slow (moving parts), fragile	Bulk/backup storage; tape archive
Flash/SSD	EEPROM cells trap electrons; no moving parts	Fast, silent, low power, durable	Costlier per GB, finite write cycles	Laptop main drive; USB stick; SD card
Optical	Laser detects pits and lands via reflected light	Very cheap, portable, long shelf life	Low capacity, slow, easily scratched	Distributing films/software; long-term disc archive

7. Challenge box (i) Virtual memory uses **secondary storage (a swap/paging file) as extra RAM** when physical RAM is full; pages are moved out to disk and back as needed. It is being used because the open applications need more memory than the installed RAM provides. (ii) The problem is **disk thrashing** — excessive paging. It is slow because **secondary storage (disk) is far slower than RAM**, so constantly swapping pages in and out wastes most of the CPU's time waiting. (iii) e.g. **Install more RAM** so fewer pages need swapping to disk (less/no paging), or **fit an SSD** so any paging that does occur is much faster than on an HDD.